

PySpark

DATA ENGINEER

Prepared by Sindhura Basavaraju - [LinkedIn](#)
Current as of September 2026.

PySpark Data Engineer

THE GOLDEN RULE Read less data → move less data → spread the work evenly → keep files healthy
→ keep big data away from the driver.

1. PySpark in 1 minute

- PySpark allows Python to work with Apache Spark, so you can use Python to process large amounts of data.
- Spark takes a large data-processing job, breaks it into smaller tasks, and runs those tasks across multiple machines at the same time.
- A DataFrame is a structured way to represent data in Spark. Think of it like a table with rows and columns, like a table in a database.
- Spark is useful when data is too large or processing is too heavy for a single machine. Instead of relying on one computer, Spark distributes the work across multiple machines.

PySpark enables parallel processing of large datasets across multiple machines instead of processing everything on a single machine.

SparkSession

SparkSession is the starting point for a Spark application. You use it to create and work with DataFrames.

DataFrame

Think of a DataFrame as a table in Spark, like a table in a database or a spreadsheet - but designed to work with large amounts of distributed data.

A DataFrame has four important parts:

- Rows → Each row represents one record.
- Columns → Each column represents a field or attribute, such as customer_id.
- Schema → Defines the structure of the DataFrame, including the column names and their data types, such as string, integer, date, or double.
- Partitions → The DataFrame is divided into smaller chunks called partitions. These partitions can be processed in parallel across multiple Spark executors.

2. Transformations, Actions & Lazy Evaluation

What is a transformation?

A transformation tells Spark what you want to do. It does not normally run immediately.

REMEMBER select, filter, withColumn, join, groupBy

What is an action?

An action tells Spark to actually run the work and return or save a result.

REMEMBER show, count, collect, write

What is lazy evaluation?

Spark waits before doing the work. It builds a plan first, improves the plan, then runs it.

REMEMBER Tell Spark first → Spark plans → Spark runs

Simple flow

filter → select → join → write = Spark builds the plan first, then executes it when the action happens.

3. Driver, Executors, Tasks, Stages

Thing	Easy meaning
Driver	The manager. It creates the plan and coordinates the job.
Executor	The worker. It processes data and runs tasks.
Task	A small piece of work done by one executor.
Stage	A group of tasks. A shuffle usually creates a new stage.

DRIVER WARNING If you use `collect()` or `toPandas()` on huge data, you can bring too much data to the driver and cause an out-of-memory error.

4. Partitions — one of the most important ideas

- A partition is a chunk of data.
- Spark can process different partitions at the same time.
- Good partitioning helps Spark spread the work.
- Too few partitions → not enough parallel work.
- Too many tiny partitions → extra overhead.

REMEMBER Partition = a chunk of data that Spark can work on in parallel.

Repartition vs Coalesce

`repartition()`

Changes the number of partitions and normally causes a shuffle.

REMEMBER Use it when you need to redistribute data.

`coalesce()`

Reduces the number of partitions with less movement when possible.

REMEMBER Good for reducing partitions before a write.

5. Narrow vs Wide Transformations

Narrow transformation

Data can stay where it is. Spark usually does not need to move data between machines.

REMEMBER filter, select → little/no shuffle

Wide transformation

Data must move between machines. This creates a shuffle.

REMEMBER groupBy, join, distinct, orderBy → shuffle

REMEMBER Wide = data moves. Data movement = expensive.

Why is shuffle expensive?

- Data moves across the network.
- Spark may write/read temporary data from disk.
- More memory and CPU are needed.
- A bad shuffle can make a job very slow.

6. Joins

Large table + very small table

A broadcast join is often a good choice. Spark sends the small table to the workers, so the large table does not need a big shuffle.

REMEMBER Broadcast the small table — but only if it is genuinely small enough.

Large table + large table

A shuffle join may be needed. Make sure the join keys are good and watch for skew.

REMEMBER Large + large → expect movement

Should I always broadcast the smaller table?

No. “Smaller” does not automatically mean “safe to broadcast.” A table can still be too large for executor memory.

REMEMBER Think about actual size and worker memory.

7. Data Skew

What is data skew?

One key has much more data than the other keys. That can make one task do most of the work while other tasks finish quickly.

REMEMBER One worker gets the giant pile.

Example

Customer A = 1,000 rows; Customer B = 1,200; Customer C = 10 million. Customer C is a hot/skewed key.

How do you fix skew?

- Broadcast a small table if the problem is a join.
- Use salting for a very skewed groupBy.
- Use AQE skew handling for supported skewed joins.
- Filter unnecessary data early.

- Check the Spark UI to find the slow task.

MEMORY

Skewed JOIN → AQE skew handling can help. Skewed GROUP BY → think salting.

8. Salting for a skewed groupBy

Problem

If one customer has 10 million rows, a normal groupBy(customer_id) may send all those rows to one partition.

Simple fix

- Add a small random salt to the hot key.
- First group by customer_id + salt.
- Then group by customer_id again to combine the partial results.

EASY EXAMPLE

Customer C becomes C_0, C_1, C_2 ... C_9. Ten workers can share the work. Then the ten results are added together.

INTERVIEW LINE

“For a skewed groupBy, I can use salting so the hot key is split across several partitions, then combine the partial results in a second aggregation.”

9. Read Less Data

Partition pruning

Spark skips whole data partitions that do not match the filter.

REMEMBER

If data is partitioned by year and you ask for 2026, skip 2024 and 2025.

Predicate pushdown

The filter is pushed closer to where the data is read, so less data may be loaded.

REMEMBER

Filter early, close to the source.

Column pruning

Spark reads only the columns it needs instead of reading every column.

REMEMBER

SELECT 3 columns → try to read 3 columns.

REMEMBER

Partition pruning = skip partitions. Predicate pushdown = push filters down. Column pruning = read fewer columns.

10. Parquet & File Layout

Why Parquet?

Parquet is a column-based file format. It works well for analytics because Spark can read only needed columns and use file statistics to skip some data.

REMEMBER

Parquet is usually a better analytics format than CSV.

Small-files problem

Millions of tiny files create lots of file-listing and task overhead. A good design keeps a sensible number of reasonably sized files.

If a table has millions of tiny files, adding more workers may not solve the real problem. Fix the file layout.

11. Cache & Persist

When should I cache?

Cache a DataFrame when it is expensive to create and you will use it more than once.

REMEMBER Reuse expensive work → cache can help.

Should I cache everything?

No. Cache uses memory/storage. Unnecessary caching can make the job worse.

REMEMBER Cache with a reason.

Cache vs checkpoint

Cache keeps data around for reuse. Checkpoint saves data to reliable storage and can cut the long dependency chain.

REMEMBER Cache = reuse. Checkpoint = break the chain.

12. UDFs

Why can Python UDFs be slow?

Spark has to move data between Spark's JVM side and Python. That extra work can slow processing.

REMEMBER Use Spark's built-in functions when possible.

13. Window Functions

Window functions let you calculate across related rows without collapsing all rows into one row.

- Latest record for each customer
- Rank customers
- Running total
- Previous/next value using lag/lead

REMEMBER groupBy reduces rows. Window functions usually keep the rows and add information to them.

14. AQE & Catalyst

Catalyst

Spark's query optimizer. It looks at the query and tries to create a better execution plan.

REMEMBER Catalyst = plan optimizer.

AQE

Adaptive Query Execution can change parts of the plan while the job is running, using information Spark learns at runtime.

REMEMBER AQE = adjust the plan using runtime information.

AQE skew join

AQE can handle certain skewed join partitions by splitting large partitions. Do not describe this as a general fix for every skewed groupBy.

15. Spark UI — Where I Look When a Job Is Slow

- Which stage is slow?
- Is there a huge shuffle?
- Is one task much slower than the others? → possible skew
- Are tasks spilling to disk?
- Are there too many tasks/files?
- Is the driver using too much memory?

The Spark UI and physical plan should be checked first to identify the bottleneck before increasing the number of workers.

16. Driver Memory vs Executor Memory

Driver memory problem

Usually happens when too much data or too much work is brought to the driver. `collect()` and `toPandas()` are common traps.

REMEMBER Big data should stay distributed.

Executor memory problem

Workers are running out of memory while processing their partitions. Large joins, big groups and bad partitioning can contribute.

REMEMBER Think: which worker is running out of memory, and why?

17. Fault Tolerance & Lineage

Spark remembers how a DataFrame was built. If a partition is lost, Spark can often rebuild the missing data from that history instead of starting the whole job again.

REMEMBER Lineage = the recipe Spark can use to rebuild lost work.

18. Structured Streaming

- Streaming means processing data as it arrives.
- A streaming query runs continuously or in repeated small batches.
- Checkpointing helps Spark remember progress.
- Watermarks help Spark manage late data and old state.

What is a watermark?

It tells Spark how long it should wait for late data. Data that arrives much later can be dropped from state depending on the query.

REMEMBER Watermark = limit how long Spark keeps waiting.

Exactly-once?

Do not promise it blindly. Exactly-once depends on the source, processing, checkpointing and sink. The whole design matters.

REMEMBER Exactly-once is end-to-end.

19. Delta & Incremental Processing

Scenario: only 2% changed, but job reads and overwrites 5 TB every night. What would you do?

Stop doing a full refresh. Find only the changed records using CDC, a last-updated timestamp, or another reliable change method. Then update the target incrementally, for example with Delta MERGE.

REMEMBER Only process what changed.

Important detail

MERGE does not mean Spark physically rewrites exactly 2% of the table. Delta works with files, so affected files may still need to be rewritten. Good file layout and target-side filtering matter.

20. Performance Mental Map

Problem	Think first
Too much data read	Partition pruning, predicate pushdown, column pruning
Too much data moved	Shuffle, join strategy, filter early
One task is very slow	Data skew
Millions of tiny files	Fix file layout / compact files
Same expensive data used many times	Cache / persist
Driver crashes	collect(), toPandas(), too much driver work
Python code is slow	Avoid Python UDF if a Spark function can do it

Reduce data read. Reduce shuffle. Spread work evenly. Keep files healthy. Keep big data away from the driver.

21. Scenario Question Bank

A groupBy is very slow. One task runs for 20 minutes while others finish in 1 minute.

Suspect data skew. Check the Spark UI. If it is a skewed aggregation, consider salting and a two-step aggregation.

A 50 MB customer lookup table is joined to a 2 TB fact table.

Consider a broadcast join if the small table fits safely in executor memory.

A query needs only 2026 data from a table partitioned by year.

Use a filter on the partition column so Spark can prune old partitions.

A table has 20 columns but the job needs only 4.

Select only the needed columns. Column pruning can reduce I/O.

A 10 TB DataFrame is sent to Pandas.

Do not do that. Keep the data distributed or aggregate/filter first.

A Python UDF is used for a simple string operation.

Replace it with a native Spark function if possible.

A job is slow because it has millions of tiny files.

Fix the file layout/compaction. More workers alone may not solve it.

A DataFrame is expensive to create and is used 5 times.

Cache/persist may help. Check memory and unpersist when no longer needed.

Only 2% of a 5 TB Delta table changes each day.

Use incremental processing and MERGE rather than full read/full overwrite.

A large-large join has one key with huge volume.

Investigate skew. Consider AQE skew join handling, better filtering, or another skew strategy.

22. Rapid-Fire questions

Is HNS the same as partition pruning?

No. HNS is about the filesystem/directory structure. Partition pruning is Spark skipping irrelevant data partitions.

Does HNS itself make Spark faster?

Not automatically. It mainly helps filesystem and metadata operations at scale.

Does groupBy always cause a shuffle?

It commonly does when data must be brought together by key. Think of groupBy as a likely shuffle operation.

Does coalesce always avoid a shuffle?

No absolute guarantees. It is mainly used to reduce partitions with less movement when possible.

Should I add more executors to every slow job?

No. First find the bottleneck.

Should I cache every DataFrame?

No. Cache when reuse makes it worthwhile.

Is broadcast join always better?

No. It is good when one side is small enough.

Is AQE skew handling the answer to skewed groupBy?

Not as a general rule. For skewed aggregation, salting is a common technique.

Does MERGE rewrite only changed rows?

Think in files, not rows. Affected files may be rewritten.

Is exactly-once always guaranteed in streaming?

No. It depends on the full source-to-sink design.

23. Simple Coding Patterns

Filter + select

```
df.filter(df.country == "AU").select("customer_id", "revenue")
```

Native function instead of simple UDF

```
from pyspark.sql.functions import upper

df.withColumn("name_upper", upper("name"))
```

Broadcast join

```
from pyspark.sql.functions import broadcast

df.join(broadcast(lookup_df), "customer_id")
```

Window: latest row

```
from pyspark.sql.window import Window
from pyspark.sql.functions import row_number, col

w = Window.partitionBy("customer_id").orderBy(col("updated_at").desc())
latest = df.withColumn("rn", row_number().over(w)).filter("rn = 1")
```

Salting idea

```
# 1. Add salt to the hot key
# 2. Aggregate by (key, salt)
# 3. Aggregate again by key
```

24. How to Answer Scenario Questions

- 1. Say what the problem is.
- 2. Say what you would check.
- 3. Say the likely fix.
- 4. Explain why the fix helps.
- 5. Say how you would measure the result.

EXAMPLE

“The job is slow because one partition is doing most of the work. I would confirm that in the Spark UI. If it is skew, I would use the right skew strategy — for example salting for a skewed groupBy. Then I would compare runtime and task balance before and after.”

25. Final One-Page Mental Map

1. TOO MUCH READ Prune partitions → push filters down → read fewer columns.

2. TOO MUCH MOVE Reduce shuffle → filter early → choose the right join.

3. WORK NOT EVEN Check skew → broadcast when suitable → salt skewed aggregations.

4. TOO MANY FILES Fix small files → keep file sizes sensible.

5. DRIVER PROBLEM Avoid collect/toPandas on big data. Keep work distributed.

**6. REPEATED
EXPENSIVE WORK** Cache/persist when reuse makes sense.

**7. ONLY SMALL
CHANGES** Use incremental processing instead of a full refresh.

REMEMBER

For Spark performance, reduce the data read, minimize unnecessary data movement, distribute work evenly across workers, maintain a healthy file layout, and keep large datasets away from the driver.

Last-minute checklist

- Transformations are lazy; actions run the job.
- Wide operations usually mean shuffle.
- Shuffle = expensive data movement.
- Skew = one key has too much data.
- Skewed groupBy → think salting.
- Small join table → consider broadcast.
- Partition pruning = skip partitions.
- Column pruning = read fewer columns.
- Parquet = good for analytics.
- collect/toPandas = driver risk.
- Cache only when reuse makes sense.
- Native Spark functions are usually better than Python UDFs.
- Slow job → check Spark UI before changing settings.
- Incremental changes → avoid full refresh.